Processing of Requests

In the previous section, we worked out how Action Dispatch routes an incoming request to the appropriate code in your application. Now let's see what happens inside that code.

Action Methods

When a controller object processes a request, it looks for a public instance method with the same name as the incoming action. If it finds one, that method is invoked. If it doesn't find one and the controller implements `method_missing`, that method is called, passing in the action name as the first parameter and an empty argument list as the second. If no method can be called, the controller looks for a template named after the current controller and action. If found, this template is rendered directly. If none of these things happens, an `AbstractController::ActionNotFound` error is generated.

Controller Environment

The controller sets up the environment for actions (and, by extension, for the views that they invoke). Many of these methods provide direct access to information contained in the URL or request:

action_name
: The name of the action currently being processed.

cookies
: The cookies associated with the request. Setting values into this object stores cookies on the browser when the response is sent. Rails support for sessions is based on cookies. We discuss sessions in *Rails Sessions*.

headers
: A hash of HTTP headers that will be used in the response. By default, `Cache-Control` is set to `no-cache`. You might want to set `Content-Type` headers for special-purpose applications. Note that you shouldn't set cookie values in the header directly—use the cookie API to do this.